Linux Security Incident Analysis (SOC Simulation)

April 19, 2026

Samuel Calderón Candela

Security Analyst

Environment:

- Attacker Machine: Kali Linux
- Victim Machine: Ubuntu Server 24.04
- Network: Controlled virtual environment

Environment Description

The analysis was conducted in a controlled laboratory environment designed to simulate a real-world cybersecurity incident.

The infrastructure consisted of two virtual machines within an internal network:

- An attacker machine running Kali Linux, used to simulate malicious activities such as unauthorized access attempts and the establishment of remote communication.
- A victim machine running Ubuntu Server 24.04, configured with the SSH service enabled (OpenSSH Server), representing a server exposed to the network.

Both machines were located on the same virtual network, allowing direct communication between them. This environment facilitated the simulation of a realistic compromise scenario, including remote access, code execution, and the establishment of a command-and-control (C2) channel.

INCIDENT TIMELINE

The incident developed through multiple phases, which were identified through the analysis of logs, processes, and network connections.

1. Initial Access (SSH Authentication Attempts):
   Multiple failed authentication attempts were detected through the SSH service, indicating a potential brute-force attack targeting the server. Subsequently, a successful login from the same IP address was recorded, confirming the compromise of valid credentials.

2. Remote Access Establishment:
   Following the successful login, the attacker obtained a session on the system and performed subsequent activities. The use of elevated privileges was observed, allowing complete control of the system.

3. Malicious Code Execution:
   The attacker created and executed a Python script located at /tmp/update.py. This location is commonly used to store temporary files and is frequently abused to conceal malicious artifacts.

4. Command-and-Control (C2) Channel Establishment:
   The executed script established an outbound connection from the victim machine to the attacker machine through port 4444, creating a remote communication channel that enabled command execution from the attacker system.

5. Remote Command Execution:
   Through the C2 channel, the attacker was able to execute commands on the compromised machine, maintaining remote control over the system.

6. Access to Sensitive Information (Exfiltration):
   Access to a file located at /tmp/contrasenias.txt was identified. The file contained simulated sensitive information. This access was performed through commands executed remotely, demonstrating the attacker's ability to query and potentially extract data from the compromised system.

Technical Evidence

During the incident analysis, various technical artifacts were collected to validate each phase of the attack.

1. SSH Authentication Logs:

The SSH service logs were analyzed using the journalctl -u ssh command, identifying multiple failed authentication attempts followed by a successful login from the IP address of the attacker machine.

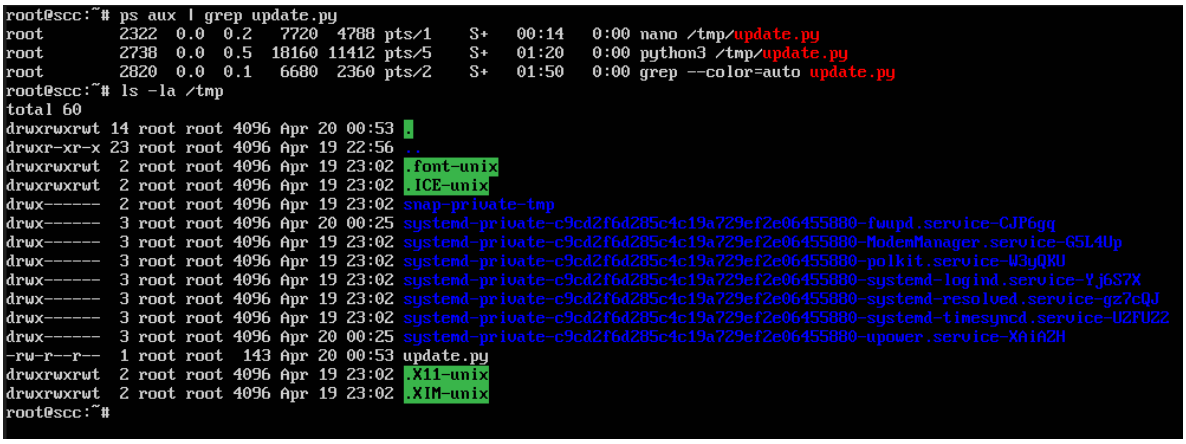
These events confirm a potential brute-force attack followed by the compromise of valid credentials.

```
root@scc:~# journalctl -u ssh | grep -E "Failed|Accepted"
Apr 19 23:37:29 scc sshd[2030]: Failed password for scc from 192.168.1.21 port 60830 ssh2
Apr 19 23:37:35 scc sshd[2030]: Failed password for scc from 192.168.1.21 port 60830 ssh2
Apr 19 23:37:42 scc sshd[2030]: Failed password for scc from 192.168.1.21 port 60830 ssh2
Apr 20 00:00:35 scc sshd[2199]: Accepted password for scc from 192.168.1.21 port 57942 ssh2
Apr 20 00:43:12 scc sshd[2423]: Accepted password for scc from 192.168.1.21 port 37718 ssh2
Apr 20 01:05:28 scc sshd[2604]: Accepted password for scc from 192.168.1.21 port 42970 ssh2
root@scc:~# _
```

2. **Suspicious Process Execution:**

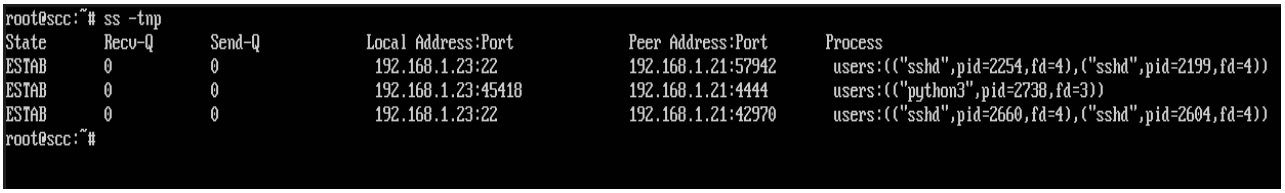
Using process monitoring tools (ps aux), a process associated with python3 executing the /tmp/update.py file was identified.

The file location and execution method are consistent with malicious behavior, as /tmp is commonly used to conceal temporary or non-persistent artifacts.



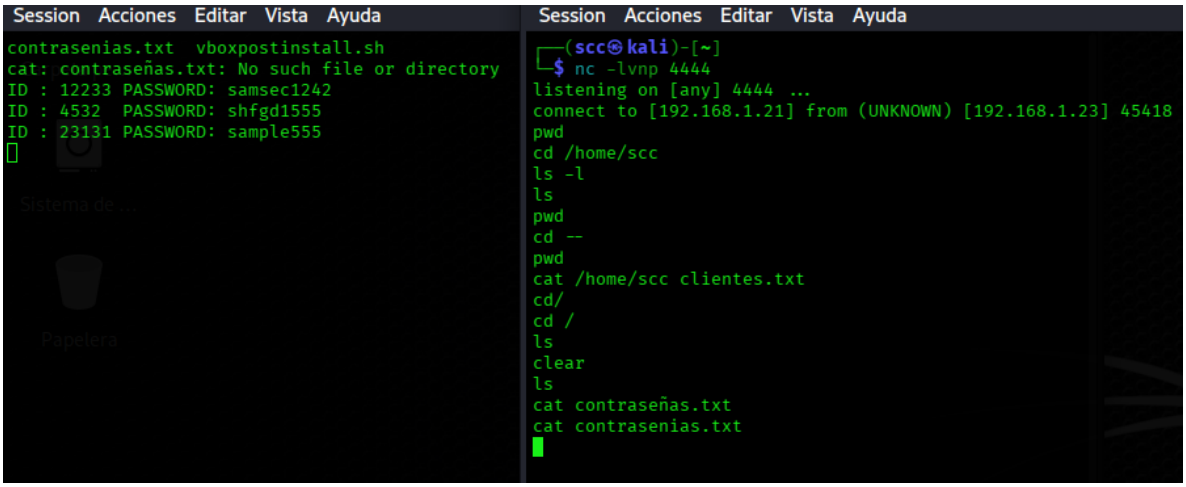
3. Through the use of the ss -tnp command, an established TCP connection was detected from the victim machine to the IP address of the attacker machine on port 4444.

   This connection was directly associated with the python3 process, providing evidence of a Command-and-Control (C2) communication channel.



4. **Evidence of Information Access:**

Access to the /tmp/contrasenias.txt file was confirmed through the execution of remote commands over the C2 channel, demonstrating the attacker's ability to interact with the system and access information stored on it.



Attacker point of vew

Indicators of Compromise (IoCs)

Based on the analysis conducted, the following Indicators of Compromise (IoCs) were identified. These indicators can be used to detect similar activity on other systems:

- Suspicious IP Address:
  The IP address of the attacker machine was identified as the source of the authentication attempts and the subsequent establishment of the C2 channel.
- Non-Standard Communication Port:
  Port 4444 was used for remote communication. This port does not correspond to a legitimate system service in the analyzed environment and can therefore be considered suspicious.
- Unusual Process:
  The python3 process was executed in association with a script located in /tmp, which represents unusual behavior in a server environment.
- Suspicious File:
  The /tmp/update.py file was present and used to establish a remote connection and execute commands on the system.
- Potentially Compromised Data File:
  The /tmp/clientes.txt file was accessed during the remote session and contained simulated sensitive information.

- Suspicious Outbound Connection:
  Network traffic was observed from the victim machine to an IP address on a non-standard port, directly associated with the identified process.

These indicators can be used to identify patterns of malicious behavior and support the creation of detection and monitoring rules in security tools.

Incident Analysis

The analyzed incident demonstrates a typical compromise chain in Linux environments exposed through remote services without adequate security controls.

Initial access was possible due to the lack of protection mechanisms on the SSH service, allowing multiple authentication attempts without restrictions. The absence of measures such as authentication attempt limits, multi-factor authentication, or active monitoring facilitated the compromise of valid credentials.

Once inside the system, the attacker was able to execute code without restrictions, indicating weaknesses in execution control and host activity monitoring. The use of the /tmp directory to host the malicious script demonstrates the exploitation of locations that are commonly overlooked during security reviews.

The establishment of a C2 communication channel over a non-standard port reflects a lack of control over outbound connections. The absence of firewall rules or traffic inspection mechanisms allowed the remote communication to take place without being detected or blocked.

Finally, access to sensitive information demonstrates the absence of adequate access controls and monitoring mechanisms for critical files. This allowed the attacker to interact freely with the system and access stored information.

Overall, the incident was enabled by a combination of weaknesses in authentication, monitoring, execution control, and network traffic management, allowing the attack to progress from initial access to potential information exfiltration.

Security Recommendations

Based on the analysis conducted, the following recommendations are proposed to mitigate risks and prevent similar incidents in the future:

1. **Harden the SSH Service:**
   Implement protection mechanisms such as authentication attempt limits, public key authentication instead of passwords, and disabling direct root login. Additionally, tools such as Fail2Ban are recommended to block IP addresses associated with multiple failed authentication attempts.
2. **Event Logging and Monitoring:**
   Establish continuous monitoring of system logs, particularly SSH service logs, to detect anomalous behavior. Integrating these logs into a SIEM would enable more effective correlation and real-time alert generation.
3. **Process Execution Control:**
   Implement controls capable of detecting and restricting script execution from unauthorized locations such as /tmp. The use of endpoint security tools (EDR) would facilitate the detection of suspicious processes.
4. **Restrict Outbound Connections:**
   Configure firewall rules to limit outbound connections to only the services required. This would help block attempts to establish C2 channels over unauthorized ports**.**
5. **Protect Sensitive Information:**
   Apply access controls to critical files and monitor their use. Implementing file access auditing would help detect unauthorized read attempts.
6. **Security Awareness and Best Practices:**
   Promote the use of strong passwords and robust authentication policies. Security training for administrators and users significantly reduces the likelihood of successful compromises.

Conclusions

The analyzed incident demonstrates how a combination of insecure configurations and a lack of monitoring controls can allow the complete compromise of a system.

Starting with initial access through the SSH service, the attacker was able to execute malicious code, establish a remote communication channel, and access sensitive information within the system. The progression of the attack highlights the importance of implementing controls at every stage, from authentication to network traffic monitoring and process execution.

The analysis enabled the identification of clear indicators of compromise and the reconstruction of the complete attack sequence, highlighting the importance of correlating logs, processes, and network connections when detecting security incidents.

This exercise reinforces the need for a proactive security approach in which prevention, detection, and response are continuously integrated to reduce the attack surface and mitigate the impact of potential threats.